
Balancing Exploration vs Exploitation

Parent program selection in ShinkaEvolve

How one sampling distribution decides whether the search climbs the current hill or goes looking for a better one.

`shinka/database/{parents, island_sampler, inspirations}.py` · `shinka/llm/prioritization.py`

Every generation buys exactly one bet

Each generation, ShinkaEvolve picks one parent program, asks an LLM to mutate it, and pays one evaluation. The distribution used to pick that parent is the entire exploration/exploitation policy.

Exploit — mutate the current best

hill-climb the strongest lineage

- + fastest score gains while the hill is climbable
- over-mines one program; stalls in local optima
- LLM keeps seeing the same code, proposes near-duplicates

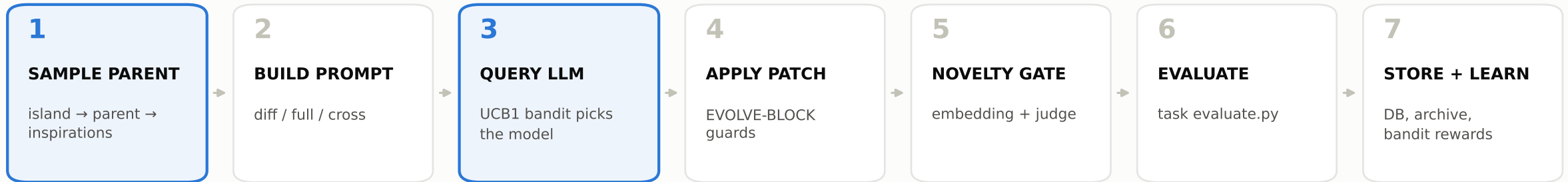
Explore — mutate something diverse

spend budget on unproven lineages

- + escapes local optima, keeps stepping-stones alive
- + novelty for the LLM to recombine later
- most evaluations don't improve the best score

ShinkaEvolve never commits to either pole. It layers three stochastic samplers — island → parent → inspirations — each with a tunable dial between the two.

Where selection sits in one generation



Two of the seven steps are explicit explore/exploit decisions:

Step 1 — which program becomes the parent (this talk's core): three nested samplers pick an island, then a parent within it, then inspiration programs shown alongside it in the prompt.

Step 3 — which LLM mutates it: an adversarial-bandit problem over the model ensemble, solved with UCB1 (slide 14). Same dilemma, different resource.

Everything downstream (patching, gating, evaluation) only measures the bet — steps 1 and 3 place it.

Structural exploration by partition

The population is split into num_islands independent sub-populations. Children always stay on the parent's island, so lineages can't collapse into one global winner. Periodic migration (every migration_interval generations, migration_rate of each island, best program protected by elitism) re-injects exploitation across islands.

island_selection_strategy		direction of the dial
uniform	pick any initialized island, equal probability	neutral (default)
equal	pick the island with the fewest programs	explore — evens out budget
proportional	Boltzmann over each island's best fitness → slide 5	exploit — feeds the winners
weighted	best fitness ÷ program count → slide 6	both — fitness pulls, crowding pushes

Defaults: num_islands = 2, migration_interval = 10, migration_rate = 0.0, island_elitism = True (shinka/database/dbase.py)

Boltzmann selection over island fitness

$$P(\text{island } i) = \frac{\exp(f_i / T)}{\sum_j \exp(f_j / T)}$$

f_i

best fitness on island i

MAX(combined_score) among the island's correct programs.

T

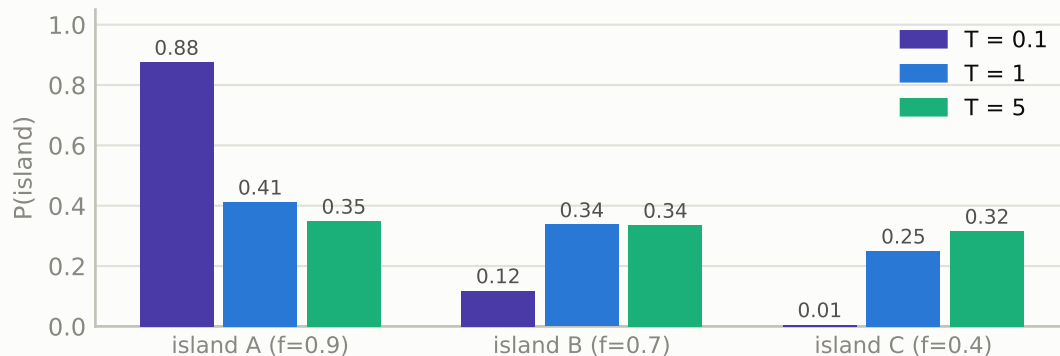
temperature (fixed at 1.0)

Divides fitness before exponentiating — the sharpness dial.

$\exp(\cdot) / \sum$

softmax normalization

Turns fitness gaps into a proper probability distribution.



Limit behavior — T is the dial

- $T \rightarrow 0$ argmax: always the best island (pure exploitation)
- $T = 1$ fitness gaps matter, but every island keeps mass
- $T \rightarrow \infty$ uniform: fitness ignored (pure exploration)

ProportionalIslandSampler — shinka/database/island_sampler.py:150

Fitness pulls, crowding pushes

$$w_i = \frac{f_i^{w_f}}{c_i^{w_c}} \quad P(\text{island } i) = \frac{w_i}{\sum_j w_j}$$

$f_i^{w_f}$

exploitation term (numerator)

Best fitness on the island.
Higher fitness → more selection mass.
Default exponent $w_f = 1$.

$c_i^{w_c}$

exploration term (denominator)

Count of correct programs — a crowding penalty. Busy islands get discounted.
Default exponent $w_c = 1$.

w_f, w_c

the two dials

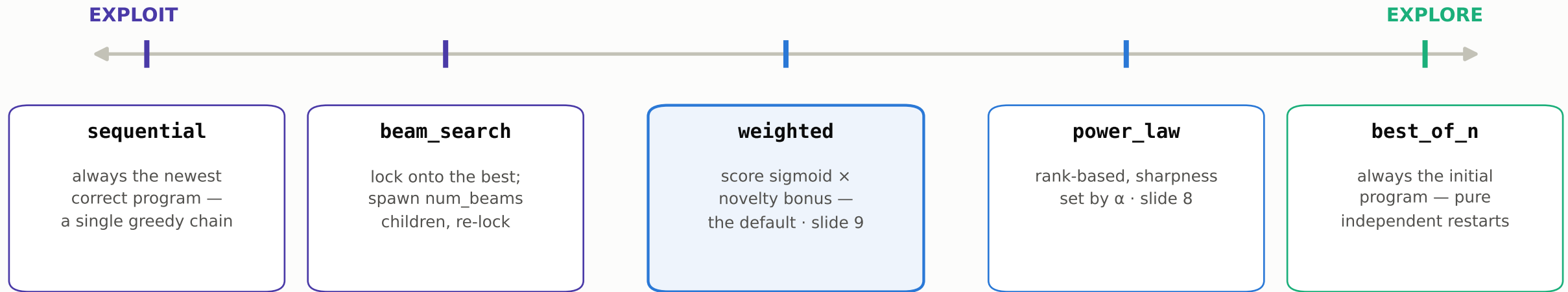
Raise w_f → greedier on fitness.
Raise w_c → stronger push toward under-explored islands.

Why divide instead of subtract?

The ratio makes the two forces compete multiplicatively: a high-fitness island loses probability as it accumulates programs, and a small island needs only modest fitness to be worth revisiting. With both exponents at 1, doubling an island's population exactly halves its weight.

WeightedIslandSampler — shinka/database/island_sampler.py:196 · all-zero weights fall back to uniform

Five strategies on one axis



All strategies draw from the island's archive — a bounded elite set (`archive_size = 40`) — and fall back to all correct programs, then the global best, then a random correct program.

If an island has no correct program at all, selection flips into FIX mode: a random broken program is sampled for repair instead (`sample_parent_with_fix_mode`).

`parent_selection_strategy` — default "weighted" · [shinka/database/parents.py](#)

One exponent sets the greed

$$P(\text{rank } k) \propto k^{-\alpha} \quad k = 1 \dots N \text{ (best first)}$$

k

rank, not raw score

Archive sorted by combined_score,
best = rank 1. Using ranks makes the
rule scale-free: only order matters.

α

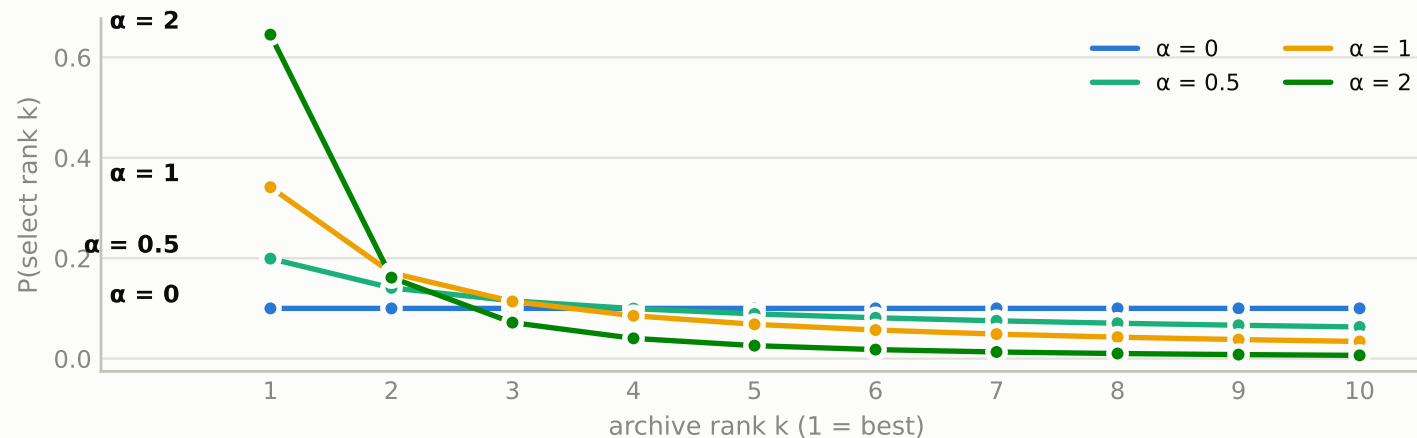
exploitation_alpha (default 1.0)

The single dial. Probabilities are
normalized after exponentiation.

$\alpha = 0 / \alpha \gg 1$

the two poles

$\alpha = 0 \rightarrow$ uniform over the archive.
Large $\alpha \rightarrow$ effectively argmax.
 $\alpha < 0$ even favors the worst.



Reading the curves

$\alpha = 1$ (default): rank 1 gets ~34%,
rank 10 still gets ~3.4% — the tail
never dies, so every archived
program keeps a path back into
the search.

sample_with_powerlaw —
shinka/database/parents.py:35

Performance × novelty, in one weight

$$P(i) = \frac{w_i}{\sum_j w_j} \quad w_i = s_i \cdot h_i$$

$$s_i \in (0, 1)$$

performance term — exploitation

Sigmoid of the program's score relative to the archive median (slide 10). Good scores saturate toward 1, bad ones toward 0.

$$h_i = \frac{1}{1 + n_i}$$

novelty term — exploration

Inverse of the children count n_i (slide 11). Every time a program is mutated, its future weight shrinks — mined-out parents fade.

Why multiply?

A product is a veto gate: a program must be both good and under-explored to keep high probability. A top scorer that has been mutated 9 times is cut to 10% of its raw weight; a mediocre program is silenced by its sigmoid no matter how novel. Addition would let either term carry a bad candidate.

WeightedSamplingStrategy — shinka/database/parents.py:380-398 · candidates = the island's archived correct programs

Robust scaling, then a sigmoid

$$s_i = \sigma\left(\lambda \cdot \frac{\alpha_i - \alpha_0}{\max(\text{MAD}, 10^{-6})}\right)$$

α_i, α_0

score vs. median

α_0 = median archive score.
Only the signed gap to the median matters.

MAD

median absolute deviation

$\text{median}(|\alpha_i - \alpha_0|)$ — a robust scale unit. Outliers can't flatten everyone else.

$\lambda = 10$

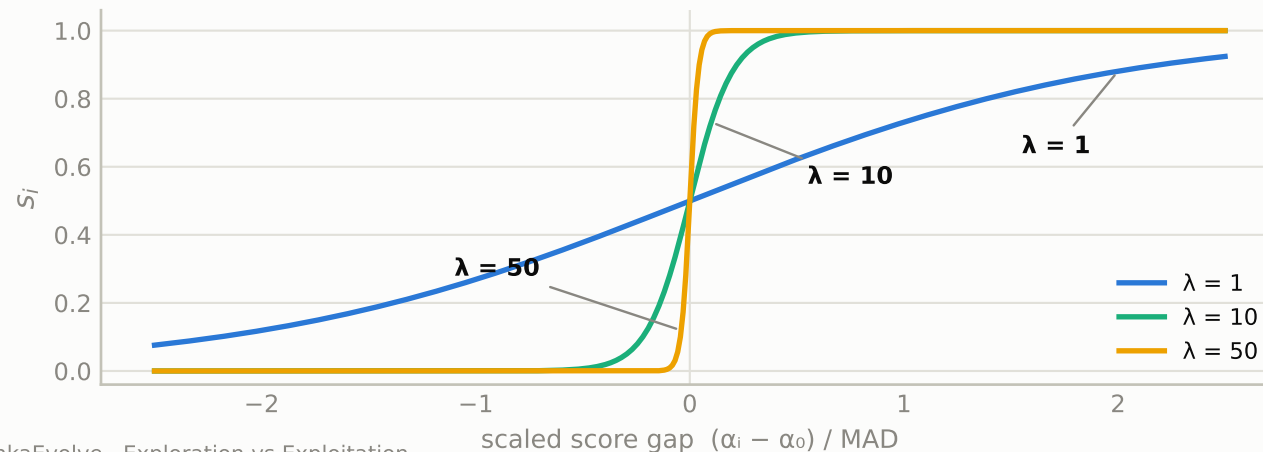
parent_selection_lambda

Sharpness. One MAD above the median already gives $\sigma(10) \approx 0.99995$.

$\sigma(x)$

stable sigmoid

Squashes to (0,1). Computed piecewise to avoid exp overflow for large $|x|$.



What $\lambda = 10$ means in practice

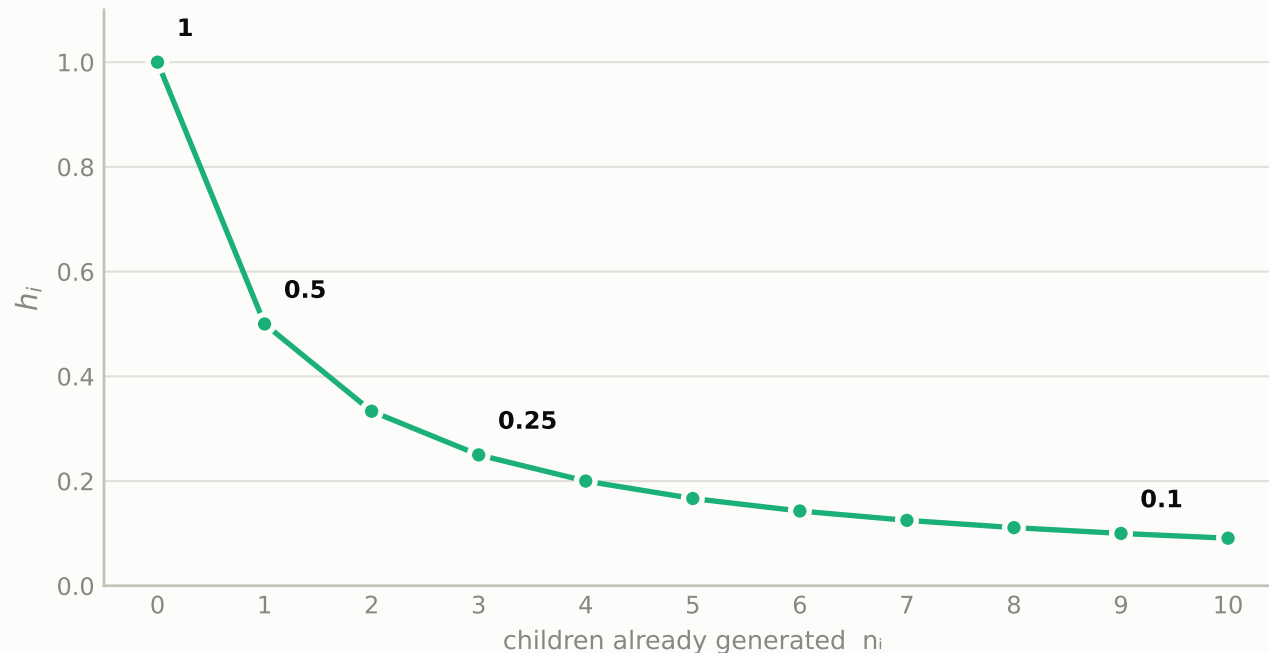
Nearly a step function at the median: above-median programs get $s_i \approx 1$, below-median ones get $s_i \approx 0$.

Exploitation sets the shortlist — the novelty term h_i picks among it.

stable_sigmoid / weights — shinka/database/parents.py:45, 368-398

Paying less for mined-out parents

$$h_i = \frac{1}{1 + n_i} \quad n_i = \# \text{ children of } i$$



A harmonic decay of attention

Fresh program ($n=0$): full weight 1.

After 1 child: $\frac{1}{2}$ — after 3: $\frac{1}{4}$ —

after 9: $\frac{1}{10}$.

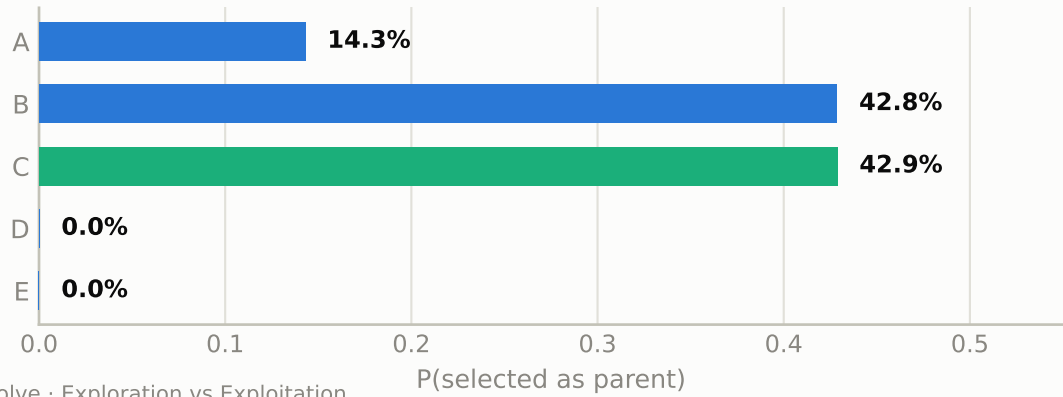
This is the only exploration force inside an island, and it is automatic: no schedule, no tuning. Selection pressure simply decays wherever budget has already been spent, pushing probability toward equally good but untried programs.

Together with s_i this reproduces the paper's parent policy (arXiv 2509.19349, §2).

Five archived programs, one draw

Archive scores $\alpha = [0.82, 0.79, 0.75, 0.70, 0.40] \rightarrow$ median $\alpha_0 = 0.75$, MAD = 0.05, $\lambda = 10$ (defaults).

program	score α_i	children n_i	s_i (perform.)	h_i (novelty)	$w_i = s_i \cdot h_i$	P(i)
A (top scorer)	0.82	5	1.000	0.167	0.167	14.3%
B	0.79	1	1.000	0.500	0.500	42.8%
C (median)	0.75	0	0.500	1.000	0.500	42.9%
D	0.70	3	4.5e-05	0.250	1.1e-05	0.0%
E (weak)	0.40	2	4.0e-31	0.333	1.3e-31	0.0%



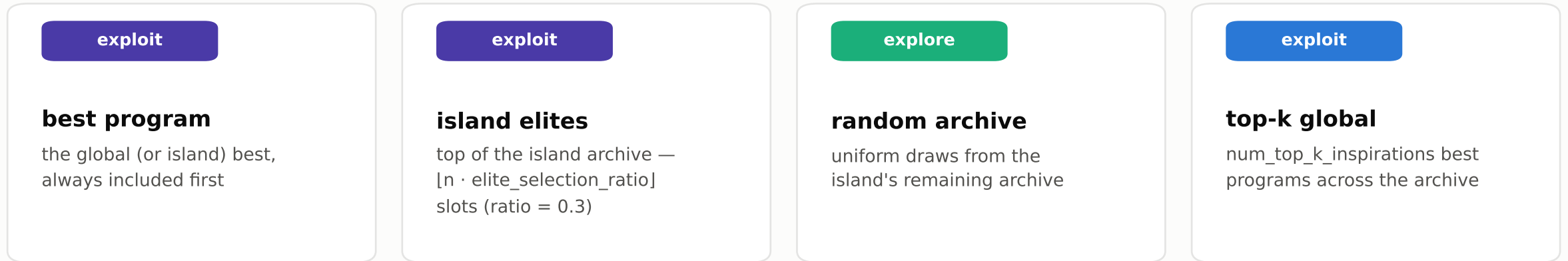
What the numbers say

C — exactly median score, never expanded — ties B and beats the top scorer A, whose five children cut its weight to 1/6.

D sits only 0.05 below the median, but $\lambda=10$ drives its sigmoid to ≈ 0 : below-median programs are effectively frozen out.

Exploration inside the prompt, not the lineage

Besides the parent, the mutation prompt shows inspiration programs. They change what the LLM sees — recombination material — without changing whose child the result is. Exploration at zero lineage cost.



Ordering is deliberate: inspirations are sorted worst → best before entering the prompt, so the strongest program sits last — closest to the model's recency bias (InspirationContextBuilder).

`enforce_island_separation = True` keeps inspiration flow island-local, preserving the island model's diversity; migration remains the only sanctioned channel between islands.

shinka/database/inspirations.py · defaults: num_archive_inspirations = 1, num_top_k_inspirations = 1

UCB1 over the model ensemble

$$\text{score}_m = \tilde{\mu}_m + c \sqrt{\frac{2 \ln t}{n_m}} \quad r = \max(\alpha_{child} - \max(\alpha_{parent}, \alpha_{base}), 0)$$

$\tilde{\mu}_m$

exploitation term

Model m's mean reward, min-max normalized over the observed range.

$c \sqrt{2 \ln t / n_m}$

exploration bonus

Optimism for rarely-trying models; shrinks as pulls n_m accumulate. $c = 1$.

$r \geq 0$

asymmetric reward

Only improvement over the parent (and global baseline) counts; regressions clip to 0.

ϵ, γ

ϵ -greedy + decay

$\epsilon = 0.2$ mass spread over non-winners; stats decay by $\gamma = 0.95$ (non-stationarity).

Why this mirrors parent selection

Parent selection asks "which program deserves the next mutation?"; the bandit asks "which LLM deserves the next query?". Both add an explicit exploration bonus to an exploitation estimate, and both decay confidence over time — because as the population evolves, which model helps is a moving target.

AsymmetricUCB.posterior — shinka/llm/prioritization.py:601-653 · unseen arms are always tried first

The dials, in one table

knob	default	what it moves	raising it →
parent_selection_strategy	"weighted"	the policy itself (see slide 7)	—
parent_selection_lambda	10.0	sigmoid sharpness around the median	↑ exploit
exploitation_alpha	1.0	power-law greed (power_law strategy)	↑ exploit
num_islands	2	independent search trajectories	↑ explore
migration_rate / interval	0.0 / 10	cross-island sharing of winners	↑ exploit
archive_size	40	how many elites stay selectable	↑ explore
elite_selection_ratio	0.3	elite share of inspiration slots	↑ exploit
exploration_coef (c) / epsilon	1.0 / 0.2	bandit optimism / forced spread	↑ explore

DatabaseConfig (shinka/database/dbase.py) and AsymmetricUCB (shinka/llm/prioritization.py)

Four ideas worth stealing

Selection is the policy

No explicit planner balances exploration and exploitation — the parent-sampling distribution is the whole policy, applied once per generation.

Multiply, don't add

$w_i = s_i \cdot h_i$ makes quality and novelty a veto gate: a parent must be both good and under-mined. Attention decays automatically wherever budget was already spent.

Be robust before being sharp

Scores are centered on the median and scaled by MAD before the $\lambda=10$ sigmoid — so the same defaults survive tasks whose scores live on wildly different scales.

The dilemma recurs at every layer

Islands (where), parents (who), inspirations (what context), and the LLM bandit (which mutator) each carry their own explore/exploit dial — tuned independently.